Projet Java : Pokémon Stadium (Version 1)

SIO 2026

25 février 2026

Table des matières

1 Objectif du Projet **2**

2 Import des données **2**
2.1 Driver MYSQL : jdbc 2
2.2 Création de la BD 2
2.3 Import des données 2
2.4 Vérifier la communication avec la base 3
2.5 Structure 3
2.6 Compilation et exécution 3

3 Fonctionnalités Principales **3**
3.1 Initialisation et Chargement des Données 3
3.2 Sélection des Joueurs 4
3.3 Sélection des Pokémon 4
3.4 Phase de Combat 4
3.5 Fin du Combat 4

4 Contraintes Techniques **5**

5 Extensions Possibles (Bonus) **5**

6 Rendu **5**

1 Objectif du Projet

L'objectif de ce projet est de développer en Java une version simplifiée de **Pokémon Stadium**. L'application permettra à un ou deux joueurs de :

— Sélectionner une équipe de six Pokémon,

— Lancer un combat entre les équipes,

— Gérer les règles fondamentales du système de combat.

Le projet s'appuiera sur une base de données relationnelle contenant les informations nécessaires : Pokémon, types, attaques, évolutions et relations d'efficacité.

2 Import des données

2.1 Driver MYSQL : jdbc

Il faut installer le driver jdbc pour communiquer avec une base de données. Sur votre VM :

1. `wget https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-8.3.0.tar.gz`
   `tar -xvf mysql-connector-j-8.3.0.tar.gz`
2. Placer le fichier **mysql-connector-j-8.3.0.tar.gz** dans un dossier lib/
3. Vous obtenez la structure suivante :

```
pokemon/
|
+-- src/        Contient le code Java
+-- lib/        Contient les bibliothèques externes
    +-- mysql-connector-j-8.3.0.jar
```

2.2 Création de la BD

Créer une base de données sur votre serveur.

```
mysql -p

create database pokemon;
use pokemon;
```

2.3 Import des données

Récupérer les fichiers sql suivants et intégrez-les à votre base.

— **pokemon_struct.sql**

— **pokemon_data.sql**

2.4 Vérifier la communication avec la base

— Récupérer le fichier **DatabaseManager.java** et vérifier que la connexion avec la base est ok.

— Créer un fichier TestSQL.java pour tester différentes requêtes SQL de récupération des données.

2.5 Structure

Vous obtenez la structure suivante :

```
pokemon/
|
+-- src/        Contient le code Java
|   +-- DatabaseManager.java
|   +-- Test.java
|
+-- lib/        Contient les bibliothèques externes
|   +-- mysql-connector-j-8.3.0.jar
|
+-- sql/        Contient les fichiers SQL de création et d'insertion
    +-- pokemon_struct.sql
    +-- pokemon_data.sql
```

2.6 Compilation et exécution

Pour compiler votre projet en prenant en compte les dépendances de librairies (ici jdbc) vous devez écrire :

```
javac -cp ..:lib/mysql-connector-j-8.3.0.jar *.java
```

Pour exécuter (ici le fichier Test.java) :

```
java -cp ..:lib/mysql-connector-j-8.3.0.jar Test
```

3 Fonctionnalités Principales

3.1 Initialisation et Chargement des Données

Au démarrage de l'application :

— Connexion à la base de données,

— Chargement de l'ensemble des Pokémon,

— Chargement des types et de leurs relations d'efficacité,

— Chargement des attaques.

Les données sont chargées une seule fois au lancement.

3.2 Sélection des Joueurs

L'application devra permettre :

— Mode 1 joueur (contre une IA simple),

— Mode 2 joueurs (local).

Chaque joueur :

— Saisit un nom,

— Dispose d'une équipe initialement vide.

3.3 Sélection des Pokémon

Chaque joueur doit :

— Consulter la liste complète des Pokémon disponibles,

— Sélectionner exactement six Pokémon,

— Créer une instance de chaque Pokémon sélectionné.

Contraintes :

— Un joueur ne peut pas sélectionner deux fois le même Pokémon,

— Chaque équipe doit contenir exactement six Pokémon.

3.4 Phase de Combat

Le combat se déroule tour par tour :

1. Chaque joueur choisit une attaque,
2. L'ordre d'action est déterminé par la statistique de vitesse,
3. Les dégâts sont calculés,
4. Les effets sont appliqués (efficacité des types, KO),
5. Si un Pokémon est KO, le suivant entre en jeu automatiquement.

Règles Simplifiées (Version 1)

— Pas d'objets,

— Pas de talents,

— Pas d'effets complexes,

— Calcul de dégâts simplifié.

Formule de dégâts simplifiée :

$$Degats = \left(\frac{Attaque}{Defense}\right) \times Puissance \times MultiplicateurType$$

3.5 Fin du Combat

Le combat se termine lorsque tous les Pokémon d'un joueur sont mis KO. Le joueur restant est déclaré vainqueur.

4 Contraintes Techniques

- Utilisation du langage Java orienté objet,
- Séparation claire des responsabilités,
- Pas de logique SQL dans les classes métier,
- Gestion centralisée des instances uniques de Type,
- Code structuré et maintenable.

5 Extensions Possibles (Bonus)

- Intelligence artificielle avancée,
- Gestion des statuts (brûlure, poison, paralysie),
- Mode tournoi,
- Interface graphique,
- Système de sauvegarde.

6 Rendu

- Code source Java complet,
- Diagramme UML,
- Documentation technique et README.md.